

```

    // Read the current time again, and calculate an
    // estimate of next frame's delta time.
    U64 end_ticks = readHiResTimer();

    dt = (F32)(end_ticks - begin_ticks)
        / (F32)getHiResTimerFrequency();

    // If dt is too large, we must have resumed from a
    // breakpoint -- frame-lock to the target rate this
    // frame.
    if (dt > 1.0f)
    {
        dt = 1.0f/30.0f;
    }

    // Use end_ticks as the new begin_ticks for next frame.
    begin_ticks = end_ticks;
}

```

8.6 Multiprocessor Game Loops

In Chapter 4, we explored the parallel computing hardware that is now ubiquitous in consumer-grade computers, mobile devices and game consoles, and we learned the mechanics of how to write *concurrent* software that takes advantage of these parallel computing resources. In this section, we'll discuss various ways in which this knowledge can be applied to a game engine's *game loop*.

8.6.1 Task Decomposition

To take advantage of parallel computing hardware, we need to *decompose* the various tasks that are performed during each iteration of our game loop into multiple subtasks, each of which can be executed in parallel. This act of decomposition transforms our software from a *sequential* program into a *concurrent* one.

There are many ways to decompose a software system for concurrency, but as we discussed in Section 4.1.3, we can place all of them roughly into one of two categories: *task parallelism* and *data parallelism*.

Task parallelism naturally fits situations in which a number of *different* things need to be done, and we opt to do them in parallel across multiple cores. For example, we might attempt to perform animation blending in parallel with collision detection during each iteration of our game loop, or we might submit

primitives to the GPU for rendering of frame N in parallel with starting to update the state of the game world for frame $N + 1$.

Data parallelism is best suited to situations in which a *single* computation needs to be performed repeatedly on a large number of data elements. The GPU is probably the best example of data parallelism in action—a GPU performs millions of per-pixel and per-vertex calculations every frame by distributing the work across a large number of processing cores that operate in parallel. However, as we'll see in the following sections, data parallelism isn't only found on the GPU—there are plenty of tasks performed by the CPU during the game loop that can benefit from data parallelism as well.

In the following sections, we'll take a look at a number of different ways to subdivide the work that's done by the game loop, some of which employ a task-parallel approach, others of which rely on data parallelism. We'll explore the pros and cons of each approach, and then see how a general-purpose *job system* can be a useful tool for converting literally any workload into a concurrent operation that can take advantage of hardware parallelism.

8.6.2 One Thread per Subsystem

One simple way to decompose our game loop for concurrency would be to assign particular engine subsystems to run in separate threads. For example, the rendering engine, the collision and physics simulation, the animation pipeline, and the audio engine could each be assigned to its own thread. A master thread would control and synchronize the operations of these secondary subsystem threads, and would also continue to handle the lion's share of the game's high-level logic (the main game loop). On a hardware platform with multiple physical CPUs, this design would allow the threaded engine subsystems to execute in parallel with one another and with the main game loop. This is a simple example of *task parallelism*; it is depicted in Figure 8.4.

There are a number of problems with the simple approach of assigning each engine subsystem to its own thread. For one thing, the number of engine subsystems probably won't match the number of cores on our gaming platform. As a result, we'll probably end up with more threads than we have cores, and some subsystem threads will need to share a core via time-slicing.

Another problem is that each engine subsystem requires a different amount of processing each frame. This means that while some threads (and their corresponding CPU cores) are highly utilized each frame, others may sit idle for large portions of the frame.

Yet another issue is that some engine subsystems *depend* on the data produced by others. For example, the rendering and audio subsystems cannot

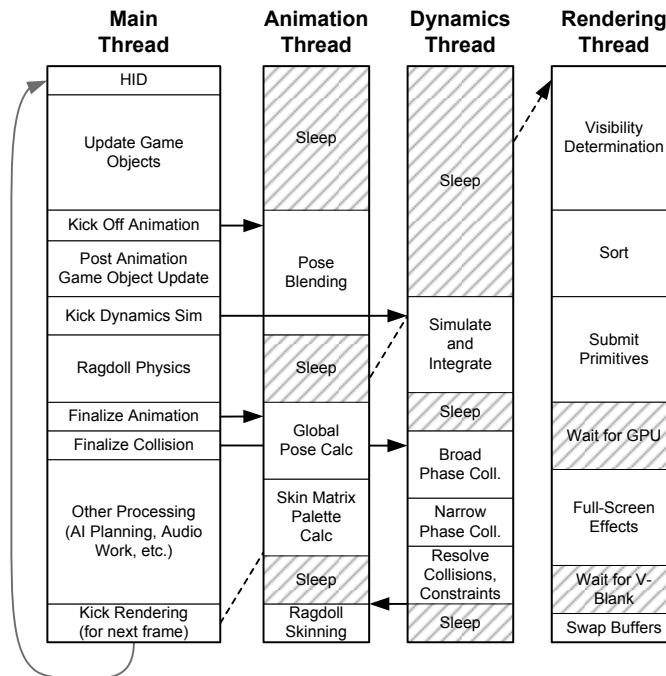


Figure 8.4. One thread per major engine subsystem.

start doing their work for frame N until the animation, collision and physics systems have completed their work for frame N . We cannot run two subsystems in parallel if they depend on one another like this.

As a result of all of these problems, attempting to assign each engine subsystem to its own thread is really not a practical design. We can do better.

8.6.3 Scatter/Gather

Many of the tasks performed during a single iteration of the game loop are data-intensive. For example, we may need to process a large number of ray cast requests, blend together a large batch of animation poses, or calculate world-space matrices for every object in a massive interactive scene. One way to take advantage of parallel computing hardware to perform these kinds of tasks is to use a divide-and-conquer approach. Instead of attempting to process 9000 ray casts one at a time on a single CPU core, we can divide the work into, say, six batches of 1500 ray casts each, and then execute one batch on each

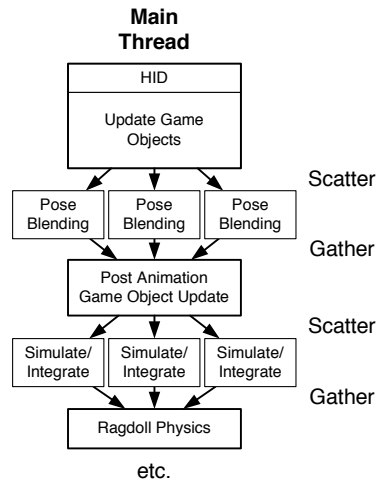


Figure 8.5. Scatter/gather used to parallelize selected CPU-intensive parts of the game loop.

of the six¹ CPU cores on our PS4 or Xbox One. This approach is a form of *data parallelism*.

In distributed systems terminology, this is known as a *scatter/gather* approach, because a unit of work is divided into smaller subunits, distributed onto multiple processing cores for execution (scatter), and then the results are combined or finalized in an appropriate manner once all workloads have been completed (gather).

8.6.3.1 Scatter/Gather in the Game Loop

In the context of our game loop, one or more scatter/gather operations might be performed, at various times during one iteration of the game loop, by the “master” game loop thread. This architecture is illustrated in Figure 8.5.

Given a dataset containing N data items that require processing, the master thread would divide the work into m batches, each containing roughly N/m elements. (The value of m would probably be determined based on the number of available cores in the system, although this may not be the case if, for example, we wish to leave some cores free for other work.) The master thread would then spawn m worker threads and provide each with a start index and count,

¹Both the PS4 and the Xbox One allow developers to access some of the processing power of a seventh core. Not all of its bandwidth is available to developers because this core is also utilized by the operating system. The eighth core in these 8-core machines is entirely off-limits. This is done to allow for the fact that some faulty cores are inevitably produced during any CPU’s fabrication process.

allowing it to process its assigned subset of the data. Each worker thread might update the data items in-place, or (usually better) it might produce output data into a separate preallocated buffer (one per worker thread).

Having successfully scattered the workload, the master thread would then be free to perform some other useful work while it waits for the worker threads to complete their tasks.

At some point later in the frame, the master thread would gather the results by waiting until all of the worker threads have terminated, perhaps using a function such as `pthread_join()`. If the worker threads have all exited, this function will return immediately; however, if any worker threads are still running, this call would have the effect of putting the master thread to sleep.

Once the gather step has been completed, the master thread could potentially combine the results in whatever manner is appropriate. For example, after blending animations together, the next step might be to calculate skinning matrices—this step would be kicked off only after all animation blending threads have completed their work. We presented an example very much like this in Section 4.4.6 when we covered thread creation and joining.

8.6.3.2 SIMD for Scatter/Gather

In Section 4.10, we explored loop *vectorization* as a means of leveraging SIMD parallelism to improve the performance of data-intensive work. This is really just another form of the scatter/gather approach, performed at a very fine level of granularity. SIMD could be used in lieu of thread-based scatter/gather, but it would likely be used in conjunction with it (with each worker thread utilizing vectorization internally to perform its work).

8.6.3.3 Making Scatter/Gather More Efficient

The scatter/gather approach is an intuitive way to distribute data-intensive work across multiple cores. However, as we've described it above, this parallelization method does suffer from one big problem—spawning threads is expensive. Spawning a thread involves a kernel call, as does joining the master thread with its workers. The kernel itself does quite a lot of set-up and tear-down work whenever threads come and go. So spawning a bunch of threads every time we want to perform a scatter/gather is impractical.

We could mitigate the costs of spawning threads by using of a pool of pre-spawned threads. Some operating systems like Windows provide an API for creating and managing a pool of threads. (See for example <https://bit.ly/2H8Chlp>.) In the absence of such an API, you can always implement a simple thread pool yourself, using *condition variables*, *semaphores*, atomic Boolean

variables, or some other mechanism in order to synchronize the activities of the threads.

We'd like our thread pool to be capable of performing a wide variety of scatter/gather operations over the course of each frame. This means that we can no longer simply spawn a bunch of threads for each scatter/gather, whose entry point function does one particular computation. Instead, each thread in the pool would have to be capable of performing any of the scatter/gather operations we might want to perform during any iteration of our game loop. We could imagine using a giant switch statement to implement this, but that idea sounds clunky, ugly and hard to maintain. Really, what we want is a general-purpose system for executing units of work concurrently, across the available cores on our target hardware.

8.6.4 Job Systems

A *job system* is a general-purpose system for executing arbitrary units of work, typically called *jobs*, across multiple cores. With a job system, a game programmer can subdivide each iteration of the game loop into a relatively large number of independent jobs, and submit them to the job system for execution. The job system maintains a queue of submitted jobs, and it schedules those jobs across the available cores, either by submitting them to be executed by worker threads in a thread pool, or by some other means. In a sense, a job system is like a custom, light-weight operating system kernel, except that instead of scheduling threads to run on the available cores, it schedules jobs.

Jobs can be arbitrarily fine-grained, and in a real game engine many are independent of one another. As shown in Figure 8.6, these facts help to maximize processor utilization. This architecture also scales up or down naturally to hardware with any number of CPU cores.

8.6.4.1 Typical Job System Interface

A typical job system provides a simple, easy-to-use API that looks very similar to the API for a threading library. There's a function for spawning a job (the equivalent of `pthread_create()`, often called *kicking* a job), a function that allows one job to wait for one or more other jobs to terminate (the equivalent of `pthread_join()`), and perhaps a way for a job to self-terminate "early" (before returning from its entry point function). A job system must also provide spin locks or mutexes of some kind for performing critical concurrent operations in an *atomic* manner. It may also provide facilities for putting jobs to sleep and waking them back up, via condition variables or some similar mechanism.

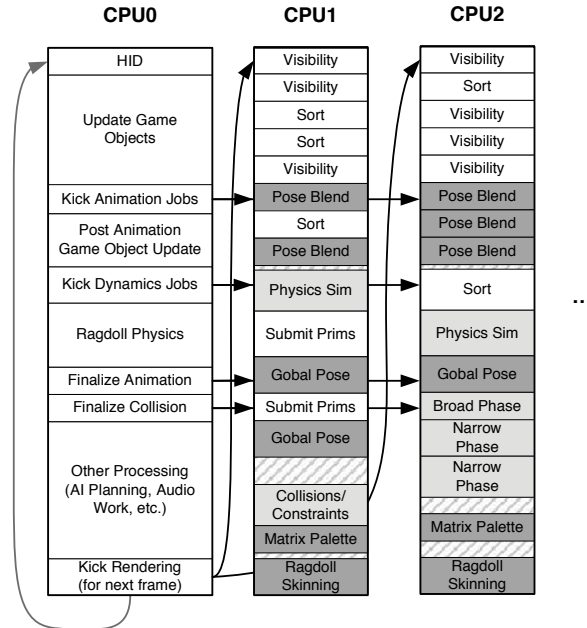


Figure 8.6. In a job model, work is broken down into fine-grained chunks that can be picked up by any available processor. This can help maximize processor utilization while providing the main game loop with improved flexibility.

To kick a job, we need to tell the job system *what* job to perform, and *how* to perform it. Normally this information is passed to the `KickJob()` function via a small data structure which we'll call a *job declaration* here.

At minimum, a job declaration must contain a pointer to the job's entry point function. It's also important to be able to pass arbitrary input parameters to the job's entry point function. This could be done in various ways, but the simplest is to provide a single parameter of type `uintptr_t`, which will be passed to the entry point function when the job actually runs. This allows us to pass simple information like a single Boolean or integer to the job with little fuss; but because a pointer can be safely cast to a `uintptr_t`, we can also use this job parameter to pass a pointer to an arbitrary data structure, that itself contains whatever input parameters the job might require.

A job system may also provide a priority mechanism, much as most thread libraries do. In this case, a priority might also be included within the job declaration.

Here's an example of a simple job declaration, along with a simple job system API:

```

namespace job
{
    // signature of all job entry points
    typedef void EntryPoint(uintptr_t param);

    // allowable priorities
    enum class Priority
    {
        LOW, NORMAL, HIGH, CRITICAL
    };

    // counter (implementation not shown)
    struct Counter ... ;
    Counter* AllocCounter();
    void FreeCounter(Counter* pCounter);

    // simple job declaration
    struct Declaration
    {
        EntryPoint*      m_pEntryPoint;
        uintptr_t        m_param;
        Priority          m_priority;
        Counter*         m_pCounter;
    };

    // kick a job
    void KickJob(const Declaration& decl);
    void KickJobs(int count, const Declaration aDecl[]);

    // wait for job to terminate (for its Counter to become zero)
    void WaitForCounter(Counter* pCounter);

    // kick jobs and wait for completion
    void KickJobAndWait(const Declaration& decl);
    void KickJobsAndWait(int count, const Declaration aDecl[]);
}

```

You'll notice there's an opaque type here called `job::Counter`. Counters allow one job to go to sleep and wait until one or more other jobs have completed executing. We'll discuss job counters in Section 8.6.4.6.

8.6.4.2 A Simple Job System Based on a Thread Pool

As we hinted at earlier, it's possible to build a job system around a pool of worker threads. It's a good idea to spawn one thread for each CPU that's present in the target machine, and use affinity to lock each thread to one core. Each worker thread sits in an infinite loop processing job requests that are

provided to it by other threads (and/or other jobs). At the top of this infinite loop, the worker thread goes to sleep (possibly via a condition variable), waiting for a job request to become available. When a request is received, the worker thread wakes up, calls the job's entry point function and passes the input parameter from the job declaration to it. When the entry point function returns, that indicates that the work is complete, so the worker thread goes back to the top of its infinite loop to execute more jobs. If none are available, it goes back to sleep, waiting for another job request to come in.

Here's what a job worker thread might look like under the hood. Please bear in mind that this is not a complete implementation; it's just for illustrative purposes.

```
namespace job
{
    void* JobWorkerThread(void*)
    {
        // keep on running jobs forever...
        while (true)
        {
            Declaration declCopy;

            // wait for a job to become available
            pthread_mutex_lock(&g_mutex);
            while (!g_ready)
            {
                pthread_cond_wait(&g_jobCv, &g_mutex);
            }

            // copy the JobDeclaration locally and
            // release our mutex lock
            declCopy = GetNextJobFromQueue();
            pthread_mutex_unlock(&g_mutex);

            // run the job
            declCopy.m_pEntryPoint(declCopy.m_param);

            // job is done! rinse and repeat...
        }
    }
}
```

8.6.4.3 A Limitation of Thread-Based Jobs

Let's imagine writing a job that updates the state of an AI-driven non-player character (NPC). The job's entry point function might look something like this:

```

void NpcThinkJob(uintparam_t param)
{
    npc* pNpc = reinterpret_cast<npc*>(param);

    pNpc->StartThinking();
    pNpc->DoSomeMoreUpdating();
    // ...

    // now let's cast a ray to see if we're aiming
    // at anything interesting -- this involves
    // kicking off another job that will run on
    // a different code (worker thread)
    RayCastHandle hRayCast = CastGunAimRay(pNpc);

    // the results of the ray cast aren't going to
    // be ready until later this frame, so let's
    // go to sleep until it's ready
    WaitForRayCast(hRayCast);

    // zzz...

    // wake up!

    // now fire my weapon, but only if the ray
    // cast indicates that we are aiming at an
    // enemy
    pNpc->TryFireWeaponAtTarget(hRayCast);

    // ...
}

```

This job seems simple enough: It performs some updates, kicks off a ray cast job (on another worker thread/core) to determine at what object the NPC is aiming his gun. The NPC then fires his weapon, but only if the ray cast reports that an enemy is in his sights.

Unfortunately this kind of job wouldn't work if we tried to run it within the simple job system that we described in the previous section. The problem is the call to `WaitForRayCast()`. In our simple thread-pool-based job system, every job must *run to completion* once it starts running. It cannot “go to sleep” waiting for the results of the ray cast, allowing other jobs to run on the worker thread, and then “wake up” later, when the ray cast results are ready.

This limitation arises because in our simple system, each running job shares the same call stack as the worker thread that invoked it. To put a job to sleep, we would need to effectively *context switch* to another job. That would involve saving off the outgoing job's call stack and registers, and then overwriting the

worker thread's call stack with the incoming job's call stack. There's no simple way to do that when using such a simple job execution method.

8.6.4.4 Jobs as Coroutines

One way to overcome this problem would be to change from a thread pool based job system to one based on *coroutines*. Recall from Section 4.4.8 that a coroutine possesses one important quality that ordinary threads do not: The ability to *yield* to another coroutine part-way through its execution, and to continue from where it left off at some later time when another coroutine yields control back to it. Coroutines can yield to one another like this because the implementation actually does swap the callstacks of the outgoing and incoming coroutines within the thread in which the coroutines are running. So unlike a purely thread-based job, a coroutine-based job could effectively “go to sleep” and allow other jobs to run while it waits for an operation such as a ray cast to complete.

8.6.4.5 Jobs as Fibers

Another way to allow jobs to sleep and yield to one another is to implement them with *fibers* instead of threads. Recall from Section 4.4.7 that the key difference between fibers and threads is that context switches between fibers are always *cooperative*, never *preemptive*. A fiber-based system begins by converting one of its threads into a fiber. The thread will continue to run that fiber until it explicitly calls `SwitchToFiber()` to explicitly *yield* control to another fiber. Just as with coroutines, the entire call stack of a fiber is saved off whenever a context switch to another fiber occurs. Fibers can even migrate from one thread to another. This makes them well suited for use in implementing a job system. Naughty Dog's job system is based on fibers.

8.6.4.6 Job Counters

If we implement our job system with coroutines or fibers, we have the ability to put a job to sleep (saving off its execution context), and to wake it back up at a future time (thus restoring its execution context). This in turn permits us to implement a *join* function for our job system—a function that causes the calling job to go to sleep, waiting until one or more other jobs have completed executing. Such a function would be roughly equivalent to `pthread_join()` in the POSIX thread library or `WaitForSingleObject()` under Windows.

One way to implement this would be to associate a *handle* with each job, just as threads have handles in most thread libraries. Waiting for a job would then amount to calling a `job::Join()` function of some kind, passing the handle of the job for which you wish to wait.

One downside of the handle-based approach is that it doesn't scale well to waiting for large numbers of jobs. Also, to wait for an individual job to complete, we'd need to poll periodically to check the status of all jobs in the system. Such polling would waste valuable CPU cycles. For these reasons, the job system API presented above introduces the concept of a *counter*, which acts a bit like a semaphore, only in reverse. Whenever a job is kicked, it can optionally be associated with a counter (provided to it via the `job::Declaration`). The act of kicking the job increments the counter, and when the job terminates the counter is decremented. Waiting for a batch of jobs, then, involves simply kicking them all off with the same counter, and then waiting until that counter reaches zero (which indicates that all jobs have completed their work.) Waiting until a counter reaches zero is much more efficient than polling the individual jobs, because the check can be made at the moment the counter is decremented. As such, a counter based system can be a performance win. Counters like this are used in the Naughty Dog job system.

8.6.4.7 Job Synchronization Primitives

Any concurrent program requires a mechanism for performing *atomic operations*, and a job system is no different. Just as a threading library provides a set of *thread synchronization primitives* such as mutexes, condition variables and semaphores, so too must a job system provide a collection of *job synchronization primitives*.

The implementation of job synchronization primitives varies somewhat depending on how the job system is actually implemented. But they're usually *not* simply wrappers around the kernel's thread synchronization primitives. To see why, consider what an OS mutex does: It puts a thread to sleep whenever the lock it's trying to acquire is already being held by another thread. If we were to implement our job system as a thread pool, then waiting on a mutex within a job would put the *entire worker thread* to sleep, not just the one job that wants to wait for the lock. Clearly this would pose a serious problem, because no jobs would be able to run on that thread's core until the thread wakes back up. Such a system would very likely be fraught with deadlock issues.

To overcome this problem, jobs could use *spin locks* instead of OS mutexes. This approach works well as long as there's not very much lock contention between threads, because in that case no job will ever busy-wait for every lock trying to obtain a lock. Naughty Dog's job system uses spin locks for most of its locking needs.

Sometimes, however, a job may experience a high-contention situation. A well-designed job system can handle this via a custom "mutex" mechanism

that can put a job to sleep while it waits for a resource to become available. Such a mutex might start by busy-waiting when a lock can't be obtained. If after a brief timeout the lock is still unavailable, the mutex could yield the coroutine or fiber to another waiting job, thereby putting the waiting job to sleep. Just as the kernel keeps track of all sleeping threads that are waiting on a mutex, our job system would need to keep track of all sleeping jobs so that it can wake them up when their mutexes free up.

8.6.4.8 Job Visualization and Profiling Tools

Once you start using a job system, the graph of running jobs and their dependencies becomes large and complex very quickly. It's a good idea to provide visualization and profiling tools with any job system.

For example, the Naughty Dog job system offers a visualization view like the one shown in Figure 8.7. In this display, you can see each of the seven cores (plus the GPU) listed vertically along the left-hand edge. Time progresses from left to right, and each logical frame is demarked with vertical markers. Along each core's timeline, thin blocks represent the various jobs that ran during the given frame. Beneath each job are additional rows of thin rectangles—these represent the call stack of the job (which functions it called, and how long each one took to run).



Figure 8.7. The job system used in *Uncharted: The Lost Legacy* (© 2017™ SIE). Created and developed by Naughty Dog, PlayStation 4) and Naughty Dog's other PS4 games offers a visualization tool that shows which jobs ran on each core over the course of a given frame. Time increases from left to right. Jobs are represented by thin boxes along the timeline of each core. The functions called by each job are shown beneath it as additional thin boxes.

Jobs are color coded by their function, so users can quickly zero in on a particular job. For example, let's say we're looking for a ray cast that's taking a particularly long time. If ray cast jobs are colored red, we can visually scan the display for all the red jobs that are wider than we'd like. Clicking on a job causes all other jobs that are not of the same type to be drawn in grey, allowing all jobs of the selected job's type to be easily seen. Also, when a job is clicked, thin lines connect it to the jobs it kicked, and to the job that kicked it. Hovering your mouse over a job, or over one of the functions in its call stack beneath it, pops up some text with the name of the job or function, and its execution time in milliseconds.

One other very useful job system feature is what I'll call a *profile trap*. Let's say an area of the game is running well at 30 FPS most of the time, but once in a while it dips to 24 FPS. We can set a trap for any frame taking longer than, say, 35 ms. Then we play the game normally. As soon as a frame taking longer than 35 ms is detected, the game is automatically paused by the trap system, and the profile display is put up on-screen. It's then possible to analyze the jobs that ran that frame to find the culprit(s) causing the slowdown.

8.6.4.9 The Naughty Dog Job System

The job system used by Naughty Dog on *The Last of Us: Remastered*, *Uncharted 4: A Thief's End* and *Uncharted: The Lost Legacy* largely follows the design of the hypothetical job systems we've discussed thus far. It is based on fibers (rather than thread pools or coroutines). It makes use of spin locks and also provides a special job mutex that can put a job to sleep while it waits for a lock. It uses counters rather than job handles to implement a *join* operation.

Let's have a look at how the fiber-based job system used in the Naughty Dog engine works. When the system first boots up, the main thread converts itself to a fiber in order to enable fibers within the process as a whole. Next, job worker threads are spawned, one for each of the seven cores available to developers on the PS4. These threads are each locked to one core via their CPU affinity settings, so we can think of these worker threads and their cores as being roughly synonymous (although in reality, other higher-priority threads do sometimes interrupt the worker threads for very brief periods during the frame). Fiber creation is slow on the PS4, so a pool of fibers is pre-spawned, along with memory blocks to serve as each fiber's call stack.

When jobs are kicked, their declarations are placed onto a queue. As cores/worker threads become free (as jobs terminate), new jobs are pulled from this queue and executed. A running job can also add more jobs to the job queue.

To execute a job, an unused fiber is pulled from the fiber pool, and the

worker thread performs a `SwitchToFiber()` to start the job running. When a job returns from its entry point function or otherwise self-terminates, the job's final act is to perform a `SwitchToFiber()` back to the job system itself. It then selects another job from the queue, and the process repeats ad infinitum.

When a job waits on a counter, the job is put to sleep and its fiber (execution context) is placed on a wait list, along with the counter it is waiting for. When this counter hits zero, the job is woken back up so it can continue where it left off. Sleeping and waking jobs is again implemented by calling `SwitchToFiber()` between the job's fiber and the job system's management fibers on each core/worker thread.

For an excellent in-depth discussion of how the Naughty Dog job system was built, and why it was built that way, check out Christian Gyrling's excellent GDC 2015 talk entitled, "Parallelizing the Naughty Dog Engine," which is available at <https://bit.ly/2H6v0J4>. Christian's slides are available at <https://bit.ly/2ETr5x9>.